

ASSIGNMENT 5

/* Floyd-Warshall algorithm: Write a program for a city planner which needs to design a traffic system that minimizes the travel time between various intersections. Each road has a different travel time, and the planner wants to ensure that the shortest routes between all intersections are identified. */

```
#include <iostream>
```

```
using namespace std;
```

```
const int INF = 9999;
```

```
void warshall(int a[10][10], int n) {  
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (a[i][k] + a[k][j] < a[i][j]) {  
                    a[i][j] = a[i][k] + a[k][j];  
                }  
            }  
        }  
    }  
}
```

```
int main() {  
    int n;  
    int a[10][10];  
    cout << "Enter number of intersections: ";  
    cin >> n;  
  
    cout << "\nEnter adjacency matrix (use " << INF  
        << " for no direct road, 0 for diagonal):\n";  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            cin >> a[i][j];  
        }  
    }  
  
    cout << "\nOriginal Matrix:\n";  
    for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < n; j++) {
            if (a[i][j] == INF) cout << "-1\t";
            else cout << a[i][j] << "\t";
        }
        cout << endl;
    }

    warshall(a, n);

    cout << "\nShortest Path Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (a[i][j] == INF) cout << "-1\t";
            else cout << a[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}

```

Enter number of intersections: 4

Enter adjacency matrix (use 9999 for no direct road, 0 for diagonal):

0

9999

6

9999

3

0

5

9999

9999

2

0

8

9999

8

6

0

Original Matrix:

0	-1	6	-1
---	----	---	----

3	0	5	-1
---	---	---	----

-1	2	0	8
----	---	---	---

-1	8	6	0
----	---	---	---

Shortest Path Matrix:

0	8	6	14
---	---	---	----

3	0	5	13
---	---	---	----

5	2	0	8
---	---	---	---

11	8	6	0
----	---	---	---